# Homework 4 Solutions

In [1]:
```
import numpy as np
import pandas as pd

import sklearn
import sklearn.model_selection
import sklearn.linear_model

import scipy.stats as stats

import matplotlib.pyplot as plt

import pygam
from pygam import s,f
import matplotlib
```

## Problem 1

Understanding the math behind splines (and how we can estimate the fit).

Where smoothing splines (for a fixed $\lambda$) optimize:

$$\min_f \sum_{i=1}^{n}(y_i - f(x_i))^2 + \lambda \int (f''(x))^2 dx$$

We will show facts about splines under the understanding that a solution to this problem would actually be a natural cubic spline (with knots at each $x_i$). Such a natural cubic spline can be expressed:

$$f(x) = \beta_0 + \sum_{k=1}^{K} \beta_k b_k(x)$$

### a)

Express $\sum_{i=1}^{n}(y_i - f(x_i))^2$ as a function of the decomposition of $f(x)$ - that is $\beta_k$s and $b_k(x)$ s.

**Solution:**

$$\begin{aligned}\sum_{i=1}^{n}(y_i - f(x_i))^2 &= ||Y - f(\underline{x})||^2 \\ &= ||Y - B\beta||^2 \\ &= (Y - B\beta)^T(Y - B\beta)\end{aligned}$$

where $B_{i,k} = b_k(x_i)$, making $B$ a $n \times K$ dimensional matrix.

## b)

Express $\int (f''(x))^2 dx$ in terms of $\beta_k$ and $b_k$ (and their integrals).

**Solution:**

$$\begin{aligned}\int (f''(x))^2 dx &= \int \Big( \sum_{k=1}^{K} \beta_k b_k''(x) \Big)^2 dx \\ &= \int \sum_{i=1}^{K} \sum_{j=1}^{K} \beta_i b_i''(x) \beta_j b_j''(x) dx \\ &= \sum_{i=1}^{K} \sum_{j=1}^{K} \beta_i \beta_j \int b_i''(x) b_j''(x) dx \\ &= \beta^T \Omega \beta\end{aligned}$$

Where $\Omega_{ij} = \int b_i''(x) b_j''(x) dx$.

## c)

Combining **part (a)** and **part (b)**:

**Solution:** So, we can find $\hat{f}$ for

$$\min_f \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \int (f''(x))^2 dx$$

if we can find $\hat{\beta}$ for

$$\min_\beta (Y - B\beta)^T (Y - B\beta) + \lambda \beta^T \Omega \beta$$

which reminds us of ridge regression and we have

$$\hat{\beta}^{\text{smoothing splines}} = (B^T B + \lambda \Omega)^{-1} B^T Y$$

---

**For those who would like to see the matrix calculus:**

$$\begin{aligned}(Y - B\beta)^T (Y - B\beta) + \lambda \beta^T \Omega \beta &= Y^T Y - 2\beta^T B^T Y + \beta^T B^T B \beta + \lambda \beta^T \Omega \beta \\ &= Y^T Y - 2\beta^T B^T Y + \beta^T (B^T B + \lambda \Omega) \beta\end{aligned}$$

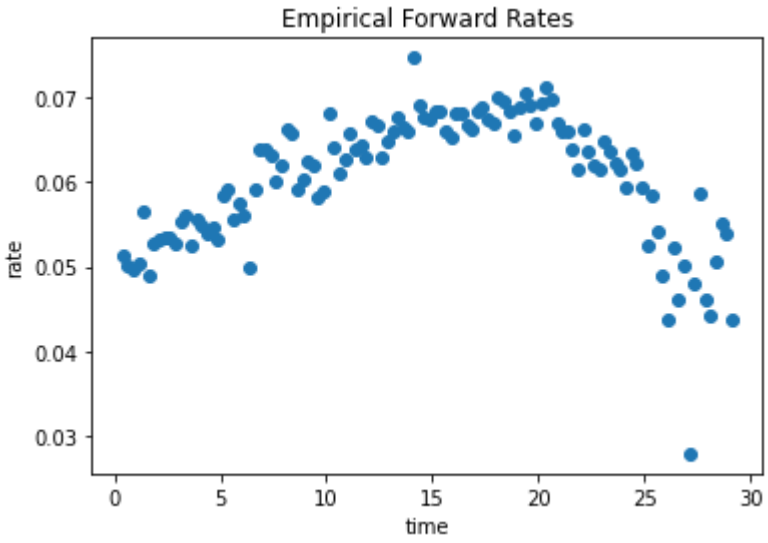
Taking the gradient with repect to $\beta$ and setting equation to $0$ we get ($\nabla_\beta(\cdot) = 0$):

# Problem 2

## a)

In [2]:
```
forward_rates = pd.read_csv("forward_rates.csv")
#forward_rates.head()

fig, ax = plt.subplots()
ax.plot(forward_rates.time, forward_rates.rate, "o")
ax.set(xlabel = "time",
       ylabel = "rate",
       title = "Empirical Forward Rates");
plt.show()
```




## b)

In [3]:
```
# fit the model
lgam_gcv = pygam.LinearGAM().fit(forward_rates["time"],
                                 forward_rates["rate"])
```

In [4]:
```
# search in (-5,5), 100 points
lgam_gcv.gridsearch(forward_rates["time"],forward_rates["rate"], lam = np.log
```

```
100% (50 of 50) |########################| Elapsed Time: 0:00:00 Time:  0:00:
00
```

Out[4]:
```
LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
   max_iter=100, scale=None, terms=s(0) + intercept, tol=0.0001,
   verbose=False)
```

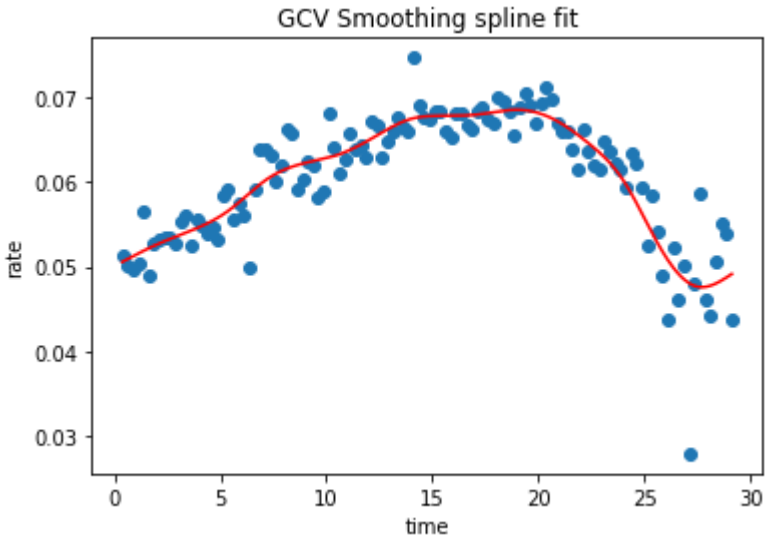
In [5]:
```
# best lambda, just to chech that it's not at the edge of the range
lgam_gcv.lam
```

Out[5]:
```
[[0.49417133613238384]]
```

**Requested Graphic:** Smoothing spline applied to our data

In [6]:
```
x_grid = np.linspace(forward_rates["time"].min(),
                     forward_rates["time"].max(),
                     num = 1000)
y_spline_gcv = lgam_gcv.predict(x_grid)

fig, ax = plt.subplots()
ax.scatter(forward_rates["time"],forward_rates["rate"])
ax.plot(x_grid, y_spline_gcv, "r")
ax.set(xlabel = "time",
       ylabel = "rate",
       title = "GCV Smoothing spline fit");
plt.show()
```




## c)

**Why would computation of yield curves be easy for cubic splines?**

Cubic splines are nice in that the functions are simple polynomials between knots. When we integrate between two bounds, we just integrate several of these polynomial pieces, each of which has a simple, closed-form solution.

# Problem 3

In [7]:
```
bike_train = pd.read_csv("bike_train.csv")
bike_test = pd.read_csv("bike_test.csv")

#I'm going to shift weathertype down by one so that gam partial dependence pl
bike_train['weathertype'] = bike_train['weathertype'] - 1
bike_test['weathertype'] = bike_test['weathertype'] - 1

bike_train.head()
```

Out[7]:

| | dayofyear | dayofweek | workday | weathertype | temp | humidity | windspeed | rentals |
|---|---|---|---|---|---|---|---|---|
| **0** | 0 | 6 | 0 | 1 | 0.344167 | 0.805833 | 0.160446 | 985 |

| | dayofyear | dayofweek | workday | weathertype | temp | humidity | windspeed | rentals |
|---|---|---|---|---|---|---|---|---|
| **1** | 1 | 0 | 0 | 1 | 0.363478 | 0.696087 | 0.248539 | 801 |
| **2** | 2 | 1 | 1 | 0 | 0.196364 | 0.437273 | 0.248309 | 1349 |
| **3** | 3 | 2 | 1 | 0 | 0.200000 | 0.590435 | 0.160296 | 1562 |

In [8]:
```
X_train = bike_train.copy()
X_train = np.array(X_train.drop(columns = ["rentals"]))
y_train = bike_train["rentals"]


X_test = bike_test.copy()
X_test = np.array(X_test.drop(columns = ["rentals"]))
y_test = bike_test["rentals"]
```

# a)

In [9]:
```
lm = sklearn.linear_model.LinearRegression().fit(X_train, y_train)

y_test_pred_lm = lm.predict(X_test) * 1.6 # recommended scaling

MSE_test_lm = np.mean((y_test_pred_lm - y_test)**2)

test_error = pd.DataFrame({"name":["MSE linear regression"],
                           "value":[MSE_test_lm]},
                          columns = ["name","value"])

test_error
```

Out[9]:

| | name | value |
|---|---|---|
| **0** | MSE linear regression | 1.295159e+06 |

# b)

You can handle the cross-validation grid in all sorts of ways for this problem. We're not going to be particular.

Here we will tune the lambdas on separate grids. The factor terms are only a few parameters, so they don't really cost us significant variance and excluding them will save computation time. All that lambda does for categorical terms is add a ridge penalty, which we don't really need badly. Using different grids for each smooth term lets us smooth some terms differently from others, depending on what the data support.

In [10]:
```
lambda_grid = np.logspace(-2, 3, 5)
lam = [lambda_grid, lambda_grid, 0,0,lambda_grid, lambda_grid, lambda_grid]
gam = pygam.LinearGAM(s(0)+s(1)+f(2)+f(3)+s(4)+s(5)+s(6),).gridsearch(X_train
```

```
100% (3125 of 3125) |####################| Elapsed Time: 0:05:38 Time:  0:05:
38
```

That took about 5 minutes on my machine. If you look at the summary below, you can see that it picks numbers inside of the range of validation parameters given (which goes from 0.01 to 1000),

which suggests that we're at least validating in the right range. We also see pretty different values of lambda for different variables, which suggests that departing from forcing them to match was probably a good idea. If this were a problem I really cared about, I might refine those grids separately around the minima for each variable to get a better fit. Another approach, which we will discuss later, is random sampling from the space of lambdas to avoid the cost and overfitting of a dense grid search.

In [11]:
```
gam.summary()
```

```
LinearGAM
============================================= ============================
============================
Distribution:                        NormalDist Effective DoF:
34.0019
Link Function:                     IdentityLink Log Likelihood:
-4818.386
Number of Samples:                         365 AIC:
9706.7757
                                               AICc:
9714.4361
                                               GCV:
258801.8004
                                               Scale:
215802.0691
                                               Pseudo R-Squared:
0.8968
============================================================================
============================
Feature Function                  Lambda               Rank         EDoF
P > x        Sig. Code
================================= ==================== ============ =========
=== ============ ============
s(0)                              [0.1778]             20           15.0
1.11e-16     ***
s(1)                              [1000.]              20           3.6
9.14e-02     .
f(2)                              [0]                  2            0.4
2.55e-01
f(3)                              [0]                  3            1.9
2.22e-06     ***
s(4)                              [56.2341]            20           3.9
1.11e-16     ***
s(5)                              [3.1623]             20           6.6
7.65e-10     ***
s(6)                              [56.2341]            20           2.6
7.20e-07     ***
intercept                                              1            0.0
1.11e-16     ***
============================================================================
============================
Significance codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

WARNING: Fitting splines and a linear function to a feature introduces a mode
l identifiability problem
         which can cause p-values to appear significant when they are not.

WARNING: p-values calculated in this manner behave correctly for un-penalized
```

```
models or models with
        known smoothing parameters, but when smoothing parameters have been
estimated, the p-values
        are typically lower than they should be, meaning that the tests reje
<ipython-input-11-dec6a6acdaaa>:1: UserWarning: KNOWN BUG: p-values computed
in this summary are likely much smaller than they should be.

Please do not make inferences based on these values!

Collaborate on a solution, and stay up to date at:
github.com/dswah/pyGAM/issues/163

  gam.summary()
```

In [12]:
```
y_test_pred_gam = gam.predict(X_test) * 1.6 # recommended scaling
MSE_test_gam = np.mean((y_test_pred_gam - y_test)**2)

test_error = pd.DataFrame({"name":["MSE linear regression",
                                   "MSE gam"],
                           "value":[MSE_test_lm,
                                    MSE_test_gam]},
                          columns = ["name","value"])

test_error
```

Out[12]:

| | name | value |
|---|---|---|
| **0** | MSE linear regression | 1.295159e+06 |
| **1** | MSE gam | 9.810118e+05 |

## c)

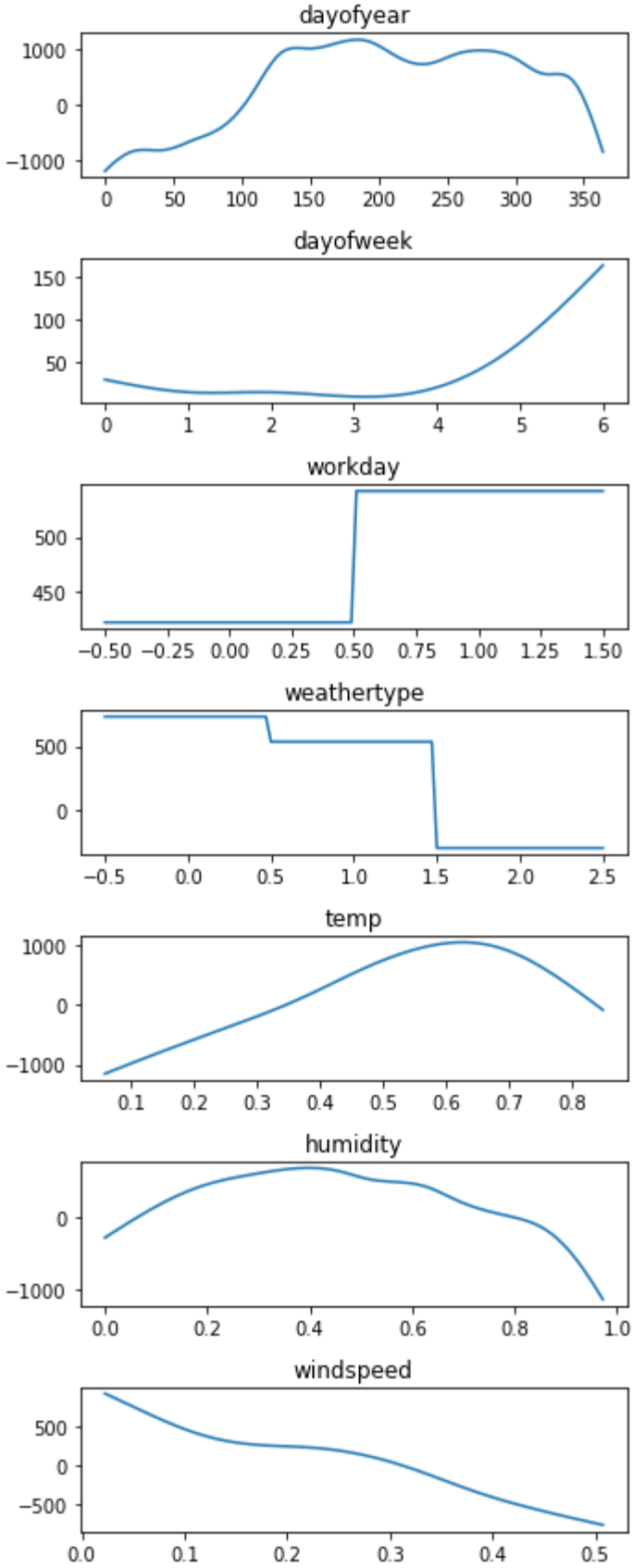
In [13]:
```
fig, axs = plt.subplots(7, 1, figsize = (5,12))
titles = list(bike_test.columns)[:7]

for i, ax in enumerate(axs):
    XX = gam.generate_X_grid(i)

    pdep, confi = gam.partial_dependence(term=i, X=XX, width=.95)

    ax.plot(XX[:, i], pdep)
    ax.set_title(titles[i])

fig.tight_layout()
plt.show()
```




You should be able to interpret the above plots yourself. Notice interesting relations between day of week and weekend. Rest of the relationships are pretty logical.

## d)

In [14]:
```
def real_loss(true, guess):
    return(np.mean(np.where(true>guess, 5*(true-guess), guess-true)))

df_out = pd.DataFrame(data =
    {"model name": ["real loss lm", "real loss gam"],
    "real loss": [real_loss(y_test, y_test_pred_lm),
                  real_loss(y_test, y_test_pred_gam)]},
                     columns = ["model name", "real loss"])
df_out
```

Out[14]:

| | model name | real loss |
|---|---|---|
| 0 | real loss lm | 2412.830285 |
| 1 | real loss gam | 2013.325371 |

## e)

In [15]:
```
five_sixth_lm_error = np.percentile(y_test - y_test_pred_lm, q = 100*5/6)
# ^ q needed in percentile
five_sixth_gam_error = np.percentile(y_test - y_test_pred_gam, q = 100*5/6)

y_test_pred_lm2 = y_test_pred_lm + five_sixth_lm_error
y_test_pred_gam2 = y_test_pred_gam + five_sixth_gam_error


df_out = pd.DataFrame(data = {"model name": ["real loss lm", "real loss gam"]
                              "real loss (prediction updated)":
                              [real_loss(y_test,y_test_pred_lm2), real_loss(y
                     columns = ["model name", "real loss (prediction updated
df_out
```

Out[15]:

| | model name | real loss (prediction updated) |
|---|---|---|
| 0 | real loss lm | 1429.453400 |
| 1 | real loss gam | 1339.225756 |

If we were really serious about minimizing this loss, we would use a version of gam that directly incorporated the alternative loss function instead of making this post-hoc correction. The interesting keyword to look into is "quantile regression" for losses of this form.

# Problem 4

## a)

$$\hat{f}(x) = \text{argmin}_{g\in\{0,1\}} \sum_{k=1}^{K} L(k,g)P(Y=k|X=x)$$
$$= \text{argmin}_{g\in\{0,1\}} \left(L_{01}\mathbf{1}_{g=1}P(Y=0|X=x) + L_{10}\mathbf{1}_{g=0}P(Y=1|X=x)\right)$$

Now consider the value of the objective for each value of g:

$$\text{objective} = \begin{cases} L_{01}P(Y=0|X=x) & \text{if } g=1 \\ L_{10}P(Y=1|X=x) & \text{if } g=0 \end{cases}$$

Since we are finding the argmin, we know that $\hat{f}(x) = 1$ exactly when $L_{01}P(Y=0|X=x) < L_{10}P(Y=1|X=x)$. Noting that $P(Y=0|X=x) = 1 - P(Y=1|X=x)$, This is equivalent to

$$L_{01} - L_{01}P(Y=1|X=x) < L_{10}P(Y=1|X=x)$$

or rearranging,

$$P(Y=1|X=x) > \frac{L_{01}}{L_{10}+L_{01}}.$$

Therefore

$$\hat{f}(x) = \begin{cases} 1 & \text{if } P(Y=1|X=x) > \frac{L_{01}}{L_{10}+L_{01}} \\ 0 & \text{if } P(Y=1|X=x) < \frac{L_{01}}{L_{10}+L_{01}} \end{cases}$$

## b)

When $L_{10} < L_{01}$, so that false positives are better than false negatives, we see that the threshold on $P(Y=1|X=x)$ is less than 1/2. This shifts us toward more false positives and fewer false negatives, as our loss function incentivizes.